University of Central Punjab

FOIT (Operating Systems)

Lab Manual: Interprocess Communication: Pipes

Topics to be Covered

- Interprocess Communication (IPC)
- Pipes
    - Theoretical concept
    - Pipe creation: `pipe()`
    - Writing to and reading from a pipe
    - Closing pipe ends
- Connecting processes through pipes
    - Parent-child and sibling communication
    - Two-way communication using two pipes
    - Pipelines of two or more processes
- Input-Output redirection
    - Duplicating descriptors: `dup()` and `dup2()`
    - Replacing a process image: the `exec()` family
    - How the shell implements `>`, `<`, `2>` and `|`

Objectives

- Students will be able to communicate between two or more related processes (parent-child or sibling) using UNIX ordinary/unnamed pipes.
- Students will be able to duplicate descriptors in a program and replace the caller process with the image of a new program.
- Students will learn how processes communicate by reading from and writing to a pipe, and how to set up two-way communication between processes.
- Students will learn how the shell runs a UNIX command made of several processes connected via pipes with input/output redirection.

System Calls

- pipe()
- read(), write()
- close()
- dup(), dup2()
- fork(), wait()
- execl(), execlp(), execv(), execvp()
- open(), perror()

1. Interprocess Communication (IPC)

In Lab 4 you saw that after `fork()` the parent and the child have **separate copies** of every variable: a change made by one is never seen by the other. Each process lives in its own private address space, so two processes cannot simply share a variable to exchange data. The operating system therefore provides special mechanisms, collectively called **Interprocess Communication (IPC)**, that let processes cooperate by exchanging data or signals.

Common UNIX/Linux IPC mechanisms:

Mechanism	Description
Unnamed (ordinary) pipe	One-way byte channel between **related** processes. Covered in this lab.
Named pipe (FIFO)	A pipe that has a name in the file system, so **unrelated** processes can use it.
Message queue	The kernel stores discrete messages that processes send and receive.
Shared memory	A region of memory mapped into two or more processes. Fastest, but needs synchronization.
Signals	Short asynchronous notifications sent to a process (for example SIGINT, SIGKILL).
Sockets	Two-way communication between processes on the same machine or across a network.

---

2. Pipes

A **pipe** is a one-way (unidirectional) data channel inside the kernel. One process writes bytes into one end and another process reads the same bytes out of the other end, in the same order they were written (**first in, first out**). It behaves like a pipe in real life: whatever goes in at one end comes out at the other.

Every `|` you type in the shell, such as `ls | wc`, creates a pipe between two processes. In this lab you will build the same thing yourself in C.

Pipe creation

Syntax

```
#include <unistd.h>

int pipe(int pfd[2]);
```

`pipe()` is passed (a pointer to) an array of two integers and fills it with **two new file descriptors**:

Element	Meaning
`pfd[0]`	Descriptor for the **read end** of the pipe.
`pfd[1]`	Descriptor for the **write end** of the pipe.

```
             +---------------------+
 write ----> |   kernel buffer     | ----> read
             +---------------------+
        (data is buffered by the kernel
         until it is read from the read end)
```

A simple way to remember it: **0 is for reading, 1 is for writing** (just like standard input is 0 and standard output is 1).

Return value

- **On success:** `pipe()` returns 0.
- **On failure:** it returns -1 and sets `errno` to indicate the reason.

Errors

Error	Meaning
EMFILE	Too many file descriptors are in use by the process.
ENFILE	The system-wide file table is full.
EFAULT	The `pfd` array is not a valid address.

Always check the return value of `pipe()`, and report failures with `perror()`.

Sample Code 1: Writing to and reading from a pipe (single process)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main()
{
    int data_processed = 0;
    int file_pipes[2];
    const char some_data[] = "123";
    char buffer[BUFSIZ + 1];
```

```
    memset(buffer, '\0', sizeof(buffer));

    if (pipe(file_pipes) == 0)
    {
        data_processed = write(file_pipes[1], some_data, strlen(some_data));
        printf("Wrote %d bytes\n", data_processed);

        data_processed = read(file_pipes[0], buffer, BUFSIZ);
        printf("Read %d bytes: %s\n", data_processed, buffer);
        exit(EXIT_SUCCESS);
    }
    exit(EXIT_FAILURE);
}
```

This program only shows how the two descriptors behave. A single process normally has no reason to use a pipe, because it could simply use a variable.

Output

```
Wrote 3 bytes
Read 3 bytes: 123
```

3. Writing to and Reading from a Pipe

The two descriptors returned by `pipe()` are ordinary file descriptors, so the ordinary `write()` and `read()` system calls are used on them.

write()

```
#include <unistd.h>

ssize_t write(int fildes, const void *buf, size_t nbyte);
```

`write()` attempts to write `nbyte` bytes from the buffer `buf` to the file associated with the open descriptor `fildes`. It returns the number of bytes actually written: **complete** (`ret == nbyte`) or **partial** (`ret < nbyte`). On error it returns -1.

read()

```
#include <unistd.h>

ssize_t read(int fildes, void *buf, size_t nbyte);
```

`read()` attempts to read up to `nbyte` bytes from the file associated with `fildes` into the buffer `buf`. On success it returns the number of bytes actually read (a non-negative integer). Otherwise it returns -1 and sets `errno`. The behavior of several processes reading from the same pipe at the same time is unspecified.

How a pipe behaves

Situation	What happens
`read()` on an **empty** pipe, write end still open	The reader is **blocked** until data arrives.
`read()` on an empty pipe, **all write ends closed**	`read()` returns **0**, which means end-of-file (EOF).
`write()` on a **full** pipe	The writer is **blocked** until a reader makes room.
`write()` when **all read ends are closed**	The writer receives the **SIGPIPE** signal (and `write()` fails with EPIPE).
`read()` asks for more bytes than are in the pipe	It returns only what is available (a partial read); it does not wait to fill the buffer.

Pipes use the kernel buffer cache just as ordinary files do. A single `write()` of up to `PIPE_BUF` bytes (at least 512 bytes, and 4096 on Linux) is **atomic**: its bytes are never mixed with bytes from another writer. If the writer does not send complete blocks, the reader may still receive partial blocks. For this reason it is good practice to buffer reads and writes on pipes (the Standard I/O library does this automatically).

Pipes carry a plain **stream of bytes**. They do not remember where one message ended and the next began, so when you send text, also send the terminating `'\0'` (for example `strlen(msg) + 1` bytes) or agree on a fixed message size.

4. Closing a Pipe End

```
#include <unistd.h>

int close(int fd);
```

`close()` closes a file descriptor so that it no longer refers to any file and may be reused. It returns 0 on success; on error it returns -1 and sets `errno`.

Closing the unused ends of a pipe is **not optional**. A pipe end is only really closed when **every** process that holds a copy of it has closed it, and the EOF rule above depends on this:

- If a reader forgets to close its own copy of the write end, `read()` will never return 0 and the reader **hangs forever** waiting for data that can never arrive.
- If all read ends are closed, a writer is killed by SIGPIPE.
- Descriptors are a limited resource, so unused ones should be released.

5. Pipes and fork()

A single process would not use a pipe. Pipes are used when **two processes** want to communicate in a one-way fashion. Once two processes already exist they cannot be connected, because there is no way for the process that made the pipe to hand a descriptor to the other one (it could pass the descriptor *number*, but that number means nothing in the other process).

The solution is inheritance. A child created with `fork()` receives a **copy of all of its parent's open descriptors**. So if the pipe is created **before** `fork()`, both processes hold valid descriptors to the same pipe.

Before fork()

```
  Parent:   pfd[1] (write) ----> [ PIPE ] ----> pfd[0] (read)
```

After fork()

```
  Parent:   pfd[1] ------+                +------> pfd[0]
                         v                |
                       [ ........ PIPE ........ ]
                         ^                |
  Child:    pfd[1] ------+                +------> pfd[0]
```

Now there are **two read ends and two write ends**. A pipe end is not closed until all copies of it are closed. Either process could write and either could read, and which process receives the data is not known. For predictable behavior one process must close its read end and the other must close its write end. The pipe is then a simple one-way pipeline again.

After closing the unused ends (parent writes, child reads)

```
  Parent:   pfd[1] ----> [ PIPE ] ----> pfd[0]   :Child
  Parent:   pfd[0] closed            pfd[1] closed :Child
```

Because processes that share a pipe must inherit it, they must be **related**: parent and child, two children of the same parent, grandparent and grandchild, and so on. An important consequence is that if one process dies, it cannot be recreated and reconnected to its pipe.

Steps to connect two processes with a pipe

1. Make the pipe with `pipe()`.
2. Call `fork()` to create the other process.
3. In the **writer** close the read end; in the **reader** close the write end. A process that does not use the pipe at all closes both ends.
4. The writer sends data with `write()` on the write end (`pfd[1]`).
5. The reader receives data with `read()` on the read end (`pfd[0]`).
6. When the writer has finished, it closes its write end. The reader then gets EOF (`read()` returns 0) and closes its read end.

Sample Code 2: The child writes, the parent reads

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define READ  0
#define WRITE 1

int main()
{
    char *phrase = "This is OS lab time";
    int fd[2], bytesread;
    char message[100];

    if (pipe(fd) == -1)
    {
        perror("pipe failed");
        exit(1);
    }

    if (fork() == 0)                    /* child, writer */
    {
        close(fd[READ]);                /* close unused end */
        write(fd[WRITE], phrase, strlen(phrase) + 1);
        close(fd[WRITE]);               /* close used end */
    }
    else                                /* parent, reader */
    {
        close(fd[WRITE]);               /* close unused end */
        bytesread = read(fd[READ], message, 100);
        printf("Read %d bytes : %s\n", bytesread, message);
        close(fd[READ]);                /* close used end */
    }
    return 0;
}
```

Output

```
Read 20 bytes : This is OS lab time
```

The parent's `read()` blocks until the child has written, so no extra synchronization is needed here. The 20 bytes are the 19 characters of the phrase plus the terminating `'\0'`.

Sample Code 3: The parent writes, the child reads until EOF

The parent sends three messages one second apart. The child keeps calling `read()` in a loop. When the parent closes its write end, `read()` returns 0 and the loop ends.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    char buf[16];
    ssize_t n;
    const char *msgs[] = {"one", "two", "three"};
    int i;

    if (pipe(fd) == -1)
    {
        perror("pipe failed");
        exit(1);
    }

    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork failed");
        exit(2);
    }
    else if (pid == 0)                    /* child, reader */
    {
        close(fd[1]);                     /* close unused write end */
        while ((n = read(fd[0], buf, sizeof(buf) - 1)) > 0)
        {
            buf[n] = '\0';
            printf("Child read %zd bytes: %s\n", n, buf);
        }
        printf("Child: read returned 0 (EOF), parent closed the pipe\n");
        close(fd[0]);
        exit(0);
    }
    else                                  /* parent, writer */
    {
```

```
        close(fd[0]);                    /* close unused read end */
        for (i = 0; i < 3; i++)
        {
            write(fd[1], msgs[i], strlen(msgs[i]) + 1);
            sleep(1);
        }
        close(fd[1]);                    /* child now gets EOF */
        wait(NULL);
    }
    return 0;
}
```

Output

```
Child read 4 bytes: one
Child read 4 bytes: two
Child read 6 bytes: three
Child: read returned 0 (EOF), parent closed the pipe
```

Experiment: remove the line `close(fd[1]);` from the **child** branch and run again. Does the child still reach the EOF message? Why or why not?

6. Two-way Communication

A pipe is one-way. If a parent must both send data to a child **and** receive a reply, it creates **two pipes** before forking. One pipe is used by the parent for writing and by the child for reading. The other is used by the child for writing and by the parent for reading.

```
  Parent ---- p2c[1] ===> [ pipe p2c ] ===> p2c[0] ---- Child
  Parent ---- c2p[0] <=== [ pipe c2p ] <=== c2p[1] ---- Child
```

After the fork, each process closes the two ends it does not use (4 `close()` calls in total, 2 in each process, plus closing the ends it used when finished).

Sample Code 4: Parent sends a string, child returns it in upper case

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    int p2c[2];                          /* parent -> child */
    int c2p[2];                          /* child  -> parent */
```

```
    char buf[100];

    if (pipe(p2c) == -1 || pipe(c2p) == -1)
    {
        perror("pipe failed");
        exit(1);
    }

    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork failed");
        exit(2);
    }
    else if (pid == 0)                    /* child */
    {
        close(p2c[1]);                    /* child only reads from p2c */
        close(c2p[0]);                    /* child only writes to c2p  */

        int n = read(p2c[0], buf, sizeof(buf) - 1);
        buf[n] = '\0';
        printf("Child  received: %s\n", buf);

        for (int i = 0; i < n; i++)       /* convert to upper case */
            if (buf[i] >= 'a' && buf[i] <= 'z')
                buf[i] = buf[i] - 'a' + 'A';

        write(c2p[1], buf, strlen(buf) + 1);
        close(p2c[0]);
        close(c2p[1]);
        exit(0);
    }
    else                                  /* parent */
    {
        close(p2c[0]);                    /* parent only writes to p2c */
        close(c2p[1]);                    /* parent only reads from c2p */

        const char *msg = "hello from parent";
        write(p2c[1], msg, strlen(msg) + 1);
        close(p2c[1]);

        int n = read(c2p[0], buf, sizeof(buf) - 1);
        buf[n] = '\0';
        printf("Parent received: %s\n", buf);
        close(c2p[0]);
        wait(NULL);
    }
```

```
    return 0;
}
```

Output

```
Child  received: hello from parent
Parent received: HELLO FROM PARENT
```

Warning: if both processes try to read first (or both write into a full pipe), each waits for the other and the program is stuck. This is a **deadlock**. Decide clearly who writes first and who reads first.

7. Input-Output Redirection

Standard descriptors

Every process starts with three open descriptors:

Descriptor	Name	Default
0	Standard input (stdin)	Keyboard
1	Standard output (stdout)	Screen
2	Standard error (stderr)	Screen

Programs such as `ls` or `sort` do not know where their output goes. They simply write to descriptor 1 and read from descriptor 0. Redirection works by changing **what descriptors 0, 1 and 2 refer to** before the program runs. It relies on two facts:

- When a process **forks**, the child inherits a copy of its parent's descriptors.
- When a process **execs**, its open descriptors (including 0, 1 and 2) remain unaffected.

The dup() and dup2() system calls

```
#include <unistd.h>

int dup (int oldfd);
int dup2(int oldfd, int newfd);
```

- `dup()` finds the **smallest free** descriptor and makes it refer to the same file (or pipe) as `oldfd`.
- `dup2()` closes `newfd` if it is open and then makes `newfd` refer to the same file as `oldfd`. After the call, `oldfd` and `newfd` are interchangeable.
- The original and the copy **share the same file offset and access mode**.
- Both return the new descriptor on success and -1 on failure.

A convenient way to read `dup2(a, b)` is: "make **b** a copy of **a**". For example `dup2(fd, 1)` means "from now on, descriptor 1 (standard output) goes wherever `fd` goes."

The exec() family (recap)

The `exec` functions replace the current process image with a new program (see Lab 4). The PID does not change, and on success the call never returns. Two of them are used in this lab:

```
int execvp(const char *file, char *const argv[]);   // arguments in an array, searched in PATH
int execlp(const char *file, const char *arg0, ..., NULL);   // arguments as a list, searched in PATH
```

The argument list must **always end with a NULL pointer**. In the variadic `execlp()` this pointer should be written as `(char *)NULL` so that it compiles cleanly with `-Wall -Werror`. By convention `arg0` is the name of the program itself.

How the shell implements redirection

For the command `ls > output` the shell does the following:

1. The parent shell calls `fork()` and waits for the child to terminate.
2. The child opens the file `output`, creating it or truncating it as necessary.
3. The child duplicates the descriptor of `output` onto descriptor 1 using `dup2()`, then closes the original descriptor. All standard output is now redirected to `output`.
4. The child calls `exec` to become the `ls` utility. Since descriptors are inherited across `exec`, all standard output of `ls` goes to `output`.
5. When the child terminates, the parent shell resumes. The parent's descriptors are unaffected, because every process has its own private descriptor table.

Sample Code 5: Redirecting the output of a command to a file

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

/* Usage: ./a.out outfile command [args...]   e.g.  ./a.out out.txt ls -l */
int main(int argc, char *argv[])
{
    int fd;

    if (argc < 3)
    {
        fprintf(stderr, "usage: %s outfile command [args]\n", argv[0]);
        exit(1);
    }

    fd = open(argv[1], O_CREAT | O_TRUNC | O_WRONLY, 0644);
    if (fd < 0)
```

```
    {
        perror("open failed");
        exit(2);
    }

    dup2(fd, 1);                            /* descriptor 1 now refers to the
file */
    close(fd);                              /* original descriptor not needed */

    execvp(argv[2], &argv[2]);              /* output of the command goes to the
file */

    perror("execvp failed");                /* reached only if exec fails */
    return 3;
}
```

Compile and run

```
$ gcc -Wall -Werror sample5.c -o sample5
$ ./sample5 output.txt ls -l
$ cat output.txt
```

Nothing is printed on the screen by `ls -l`, because its standard output now refers to `output.txt`.
This is equivalent to the shell command `ls -l > output.txt`.

Redirection is not limited to standard output:

Shell syntax	Meaning	Descriptor work in C
`cmd > file`	stdout to a file (truncate)	`open(file, O_CREAT|O_TRUNC|O_WRONLY, 0644)` `dup2(fd, 1)`
`cmd >> file`	stdout appended to a file	`open(file, O_CREAT|O_APPEND|O_WRONLY, 0644)` `dup2(fd, 1)`
`cmd < file`	stdin read from a file	`open(file, O_RDONLY)` `dup2(fd, 0)`
`cmd 2> file`	stderr to a file	`open(file, O_CREAT|O_TRUNC|O_WRONLY, 0644)` `dup2(fd, 2)`

Remember to `#include <fcntl.h>` for `open()` and its flags.

8. Connecting Two Programs with a Pipe

The UNIX shell uses unnamed pipes to build pipelines. It uses a trick similar to redirection to connect the standard output of one process to the standard input of another.

To implement `ls | wc` the shell creates a pipe and then forks. Suppose the write end of the pipe is descriptor 3 and the read end is descriptor 4. But `ls` writes to descriptor 1 and `wc` reads from descriptor 0. How do these get matched up? After `exec`, the new program has no way of knowing which descriptors belong to the pipe. The answer is `dup2()`: make descriptor 1 a copy of the pipe's write end, make descriptor 0 a copy of the pipe's read end, then close the originals.

Descriptor tables

Stage	Process writing (`ls`)	Process reading (`wc`)
After pipe() and fork()	0 stdin, 1 stdout, 2 stderr 3 pipe write end, 4 pipe read end	0 stdin, 1 stdout, 2 stderr 3 pipe write end, 4 pipe read end
After dup2()	`dup2(3, 1)`: **1** is now the pipe write end	`dup2(4, 0)`: **0** is now the pipe read end
After close()	3 and 4 closed. Only 1 is the pipe.	3 and 4 closed. Only 0 is the pipe.
After exec()	`ls` writes to 1, and the data enters the pipe	`wc` reads from 0, and the data comes from the pipe

Sample Code 6: Connecting the output of one program to the input of another

The program receives two program names on the command line and runs them as `first | second`.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

#define READ  0
#define WRITE 1

/* Usage: ./a.out writer_program reader_program    e.g.  ./a.out who wc */
int main(int argc, char *argv[])
{
    int pid, fd[2];

    if (argc < 3)
    {
        fprintf(stderr, "usage: %s writer reader\n", argv[0]);
        exit(1);
    }
    if (pipe(fd) == -1)
```

```
    {
        perror("pipe failed");
        exit(1);
    }
    if ((pid = fork()) < 0)
    {
        perror("fork failed");
        exit(2);
    }
    if (pid != 0)                           /* parent, writer */
    {
        close(fd[READ]);                    /* close unused end */
        dup2(fd[WRITE], 1);                 /* duplicate used end to standard
out */
        close(fd[WRITE]);                   /* close used end */
        execlp(argv[1], argv[1], (char *)NULL);   /* execute writer program */
        perror("exec failed");
        exit(3);
    }
    else                                    /* child, reader */
    {
        close(fd[WRITE]);                   /* close unused end */
        dup2(fd[READ], 0);                  /* duplicate used end to standard
input */
        close(fd[READ]);                    /* close used end */
        execlp(argv[2], argv[2], (char *)NULL);   /* execute reader program */
        perror("exec failed");
        exit(3);
    }
    return 0;
}
```

Run the above program as

```
$ gcc -Wall -Werror sample6.c -o sample6
$ ./sample6 who wc
```

This behaves like the shell command `who | wc`. Try other pairs, such as `./sample6 ls wc` or `./sample6 date wc`.

Why does the program close `fd[READ]` and `fd[WRITE]` after the `dup2()` call? If the reader (`wc`) kept an extra copy of the write end open, it would never see EOF and would wait forever.

9. Pipelines of Three or More Processes

A pipeline may consist of three or more processes, for example `ps | sed 1d | wc -l`. There are several ways to organize this:

- The parent can fork twice (or more) to give **sibling** children.
- The parent can fork once and the child can fork once, giving a parent, child and grandchild.
- The parent can create **all pipes before any forking**. After a fork, each process holds all the ends, so each process must close every end it does not need. With two pipes and two processes there are 8 open ends (2 processes x 2 ends x 2 pipes); most of them must be closed so that each pipe ends up with only one reader and one writer.
- As many ends as possible may be closed **before** the fork. This minimizes the number of `close()` calls needed afterwards.

For a pipeline of n processes you need **n - 1 pipes**. The first process only writes into a pipe, the last process only reads from a pipe, and each process in the middle reads from the previous pipe and writes into the next one.

Sample Code 7: Implementing `ls | grep ".c" | wc -l` with two pipes

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

/* Implements:  ls | grep ".c" | wc -l   using two pipes and three children */
int main()
{
    int p1[2], p2[2];

    if (pipe(p1) == -1 || pipe(p2) == -1)
    {
        perror("pipe failed");
        exit(1);
    }

    if (fork() == 0)                    /* child 1: ls */
    {
        dup2(p1[1], 1);                 /* stdout -> pipe 1 */
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);     /* not used by this child */
        execlp("ls", "ls", (char *)NULL);
        perror("exec ls failed");
        exit(2);
    }
    if (fork() == 0)                    /* child 2: grep */
    {
        dup2(p1[0], 0);                 /* stdin  <- pipe 1 */
        dup2(p2[1], 1);                 /* stdout -> pipe 2 */
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("grep", "grep", ".c", (char *)NULL);
```

```
        perror("exec grep failed");
        exit(2);
    }
    if (fork() == 0)                    /* child 3: wc -l */
    {
        dup2(p2[0], 0);                 /* stdin <- pipe 2 */
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc failed");
        exit(2);
    }

    /* parent must close every end, otherwise the readers never see EOF */
    close(p1[0]); close(p1[1]);
    close(p2[0]); close(p2[1]);
    wait(NULL); wait(NULL); wait(NULL);
    return 0;
}
```

Notice the two rules that make every pipeline work: **every** process closes **all** pipe ends it does not use, and the **parent also closes all of its ends** before waiting. If the parent kept a write end open, the last process (`wc`) would never see EOF.

Practice Questions

Part A: Predict the Output / Short Answers

Question 1

What does this program print?

```
int main()
{
    int fd[2];
    char buf[20] = {0};
    pipe(fd);
    write(fd[1], "ABCDE", 5);
    int n = read(fd[0], buf, 20);
    printf("%d %s\n", n, buf);
    return 0;
}
```

Question 2

The child writes "hi" into the pipe and exits. Will the parent's loop ever finish? Explain why, and say how it can be fixed.

```
int main()
{
    int fd[2];
    char c;
    pipe(fd);
    if (fork() == 0)
    {
        close(fd[0]);
        write(fd[1], "hi", 2);
        close(fd[1]);
        return 0;
    }
    while (read(fd[0], &c, 1) > 0)
        printf("%c", c);
    return 0;
}
```

Question 3

Where does the output of `ls` go in the following code? Which line will never run if `execlp` succeeds?

```
int fd = open("list.txt", O_CREAT | O_TRUNC | O_WRONLY, 0644);
dup2(fd, 1);
close(fd);
execlp("ls", "ls", "-l", (char *)NULL);
printf("done\n");
```

Question 4

A parent creates two pipes and then forks once. How many pipe descriptors are open in total (parent and child together) right after the fork, before any `close()` calls? How many must be closed so that each pipe has exactly one reader and one writer?

Question 5

Explain why two unrelated processes (for example two programs started in different terminals) cannot communicate using an unnamed pipe.

Question 6

What does `read()` return when the pipe is empty and (a) the write end is still open in some process, (b) every write end is closed?

Part B: Programming Tasks

All codes must be compiled using the `-Wall -Werror` flags and must check the return values of `pipe()` and `fork()`.

Task 1: Reverse a string Write a C program in which the parent receives a string from the command line (`argv[1]`) and sends it to its child through a pipe. The child reads the string, reverses it and prints the reversed string on the terminal. The parent must wait for the child to finish.

Task 2: Prime check (two-way) Write a C program that takes an integer n from the command line. The parent sends n to the child through one pipe. The child checks whether n is prime and sends back the answer (1 or 0) through a second pipe. The parent prints "n is prime" or "n is not prime". Test with n = 7, 15 and 97.

Task 3: Sum and minimum of an array Write a C program in which the parent process receives an integer array through command-line arguments and passes it to each of its two child processes using a pipe. Child 1 will find the **sum** of the array elements, and Child 2 will find the **minimum** of the array elements. Each child returns its result back to the parent process using a separate pipe. The parent process prints the min and sum values on the terminal.

HINT: You need to create 4 separate pipes for this task.

Write C programs to implement the given UNIX commands (Tasks 4 to 6). Do not use `system()`, and do not use the shell to do the redirection: use `open()`, `dup2()`, `pipe()`, `fork()` and `exec` yourself.

Task 4

```
$ ls / -l > input.txt
```

Task 5

```
$ grep "m" < input.txt | sort > output.txt
```

Here three redirections or processes are involved: the input of `grep` comes from a file, the output of `grep` goes into a pipe, the pipe feeds `sort`, and the output of `sort` goes to a file.

Task 6

```
$ ls / -lR 2> error.txt | grep "Nov" | sort | less
```

Here the standard error of `ls` is redirected to `error.txt` (descriptor 2), while its standard output goes into a pipe. The pipeline contains four processes and three pipes. Take a screenshot of the final output and of the contents of `error.txt`.